

TelemetriaMqtt v2 — Checkpoint 6

Dashboards Grafana — caminho histórico

Projeto de Telemetria FSAE

16 de setembro de 2026

Abstract

Este documento detalha o sexto checkpoint do projeto: colocar o Grafana no ar como container Docker, com o datasource InfluxDB e um dashboard de análise histórica provisionados inteiramente como código – nenhuma configuração manual pela interface web.

Contents

1	Contexto e objetivo	1
2	Estrutura adicionada	1
2.1	Datasource	2
2.2	Dashboard gerado por script	2
3	Testes	3
3.1	Resultado da execução	3
4	Validação manual	3
5	Nota sobre metodologia	3
6	Resumo do que ficou pronto	4
7	Próximos passos	4

1 Contexto e objetivo

O caminho histórico da arquitetura (definida no `SCOPE.md`) termina em dashboards Grafana consultando o InfluxDB (colocado no ar no checkpoint 5). A decisão de arquitetura relevante aqui: assim como o broker e o banco, o Grafana também é provisionado **como código** – datasource e dashboard são arquivos versionados no repositório, não cliques numa interface que ninguém mais consegue reproduzir. É a mesma razão pela qual o projeto rejeitou Node-RED no planejamento inicial.

2 Estrutura adicionada

Listing 1: Arquivos novos

```
1 infra/  
2   docker-compose.yml          (+ serviço grafana)  
3 grafana/
```

```

4     generate_dashboard.py                    (gera o dashboard a partir de uma
        lista)
5     dashboards/
6     historico.json                        (gerado, não editado à mão)
7     provisioning/
8     datasources/influxdb.yml
9     dashboards/dashboards.yml

```

2.1 Datasource

Listing 2: infra/grafana/provisioning/datasources/influxdb.yml

```

1  datasources:
2    - name: InfluxDB
3      uid: influxdb-main
4      type: influxdb
5      access: proxy
6      url: http://influxdb:8086
7      isDefault: true
8      jsonData:
9        version: Flux
10       organization: $INFLUXDB_INIT_ORG
11       defaultBucket: $INFLUXDB_INIT_BUCKET
12     secureJsonData:
13       token: $INFLUXDB_INIT_ADMIN_TOKEN

```

Dois pontos relevantes: a linguagem de consulta é **Flux** (não InfluxQL, a linguagem legada), consistente com o InfluxDB 2.x; e as credenciais vêm das **mesmas** variáveis de ambiente já usadas pelo próprio serviço InfluxDB (`infra/.env`) – o Grafana suporta expansão de variável de ambiente diretamente nos arquivos de provisionamento, então não existe uma segunda cópia das credenciais em lugar nenhum. O uid fixo (`influxdb-main`) permite que o dashboard referencie esse datasource diretamente, sem precisar descobrir o UID em tempo de execução.

2.2 Dashboard gerado por script

Em vez de editar o JSON do dashboard à mão (o schema do Grafana é grande e repetitivo – cada painel tem dezenas de campos), um script Python (`generate_dashboard.py`) gera o arquivo a partir de uma lista declarativa de painéis:

Listing 3: Trecho de `generate_dashboard.py`

```

1  PANELS = [
2      {"title": "RPM", "fields": ["rpm"], "unit": "none"},
3      {"title": "Acelerador / Lambda / MAP",
4       "fields": ["throttlePosition", "lambda", "map"], "unit": "none"},
5      {"title": "Temperaturas ( C )",
6       "fields": ["engineTemp", "airTemp"], "unit": "celsius"},
7      {"title": "Correntes (A)",
8       "fields": ["ventoinhaCurrent", "bombaCurrent", ...], "unit": "amp"},
9      {"title": "Velocidade GPS (km/h)", "fields": ["gpsSpeed"], ...},
10     {"title": "Tensão da bateria (V)", "fields": ["extBattery"], ...},
11     {"title": "Estados digitais",
12      "fields": ["bombaInput", "giratoriaInput"], "unit": "none"},
13 ]

```

Os 20 sinais de telemetria (mesma lista do `mock_publisher/firmware`) foram agrupados em 7 painéis por categoria, em vez de 20 painéis individuais – mais legível pra uma primeira análise histórica. Cada painel vira uma consulta Flux gerada automaticamente:

Listing 4: Query Flux gerada pro painel "RPM"

```
1 from(bucket: "telemetria")
2   |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
3   |> filter(fn: (r) => r._measurement == "telemetry")
4   |> filter(fn: (r) => r._field == "rpm")
```

Acoplamento a documentar: o nome do bucket ("telemetria") está fixo nas queries geradas – o Flux não herda automaticamente o `defaultBucket` configurado no `datasource`. Se o bucket for renomeado no futuro, o dashboard precisa ser regenerado (`python3 generate_dashboard.py > dashboards/historico.json`), não só o `datasource` reconfigurado.

3 Testes

Quatro testes validam este checkpoint contra um Grafana real (não mockado):

Teste 1 `test_docker_compose_up_starts_a_working_grafana` – sobe `influxdb+grafana` via `docker-compose.yml` real, confirma: o `healthcheck` responde; o `datasource` provisionado tem o tipo, organização e bucket corretos; o dashboard `telemetria-historico` aparece na busca com exatamente 7 painéis.

Teste 2 `test_grafana_queries_real_data_through_provisioned_datasource` – a validação mais forte do checkpoint: escreve um ponto no InfluxDB usando o `make_influx_writer` de produção, e consulta de volta através da **própria API do Grafana** (`/api/ds/query`, o mesmo endpoint que os painéis do dashboard usam internamente) – não direto no InfluxDB. Confirma que a rede entre os containers do `compose`, a autenticação do `datasource` e a linguagem Flux configurada funcionam de ponta a ponta, não só que "a configuração parece certa" olhando os arquivos.

3.1 Resultado da execução

Listing 5: Saída real de `pytest -v`

```
1 test_docker_compose.py::test_docker_compose_up_starts_a_working_grafana
  PASSED
2 test_docker_compose.py::
  test_grafana_queries_real_data_through_provisioned_datasource PASSED
```

4 Validação manual

Antes mesmo de escrever os testes automatizados, o comportamento foi confirmado manualmente via `curl` contra a API REST do Grafana: `/api/health`, `/api/datasources` (confirmando organização/bucket corretos), `/api/search` e `/api/dashboards/uid/telemetria-historico` (confirmando os 7 painéis), e por fim uma consulta real via `/api/ds/query` depois de publicar dados reais através do pipeline completo (`mock publisher` → `broker` → `ingestão`), retornando os pontos de RPM esperados.

5 Nota sobre metodologia

Este checkpoint é o exemplo mais claro, até agora, de uma inversão da ordem de TDD no projeto: toda a infraestrutura (serviço no `compose`, `datasource`, dashboard) foi construída primeiro, validada manualmente via `curl`, e só depois os testes automatizados foram escritos – essencialmente codificando um comportamento que já tinha sido confirmado, em vez de guiar a implementação. Identificado numa revisão de processo com o usuário logo após este checkpoint ser fechado. A

decisão tomada foi não refazer os checkpoints 5 e 6 (a cobertura de teste já é real, contra infraestrutura de verdade, e passa) – mas a partir do checkpoint 7 o teste volta a vir primeiro, inclusive para infraestrutura declarativa como esta.

6 Resumo do que ficou pronto

- Grafana (`grafana-oss:13.0.2`) real, provisionado inteiramente como código.
- Datasource InfluxDB (Flux) com credenciais compartilhadas via variável de ambiente, sem duplicação de segredo.
- Dashboard de 7 painéis, gerado por script a partir de uma lista declarativa, cobrindo os 20 sinais de telemetria.
- Validação de ponta a ponta confirmada: dado escrito no InfluxDB é visível através da própria API de consulta do Grafana.

7 Próximos passos

Checkpoint 7: dashboard Grafana ao vivo, usando Grafana Live + o plugin de datasource MQTT, pro caminho de visualização em tempo real (sem depender do InfluxDB) definido na arquitetura original do projeto.